Dataset Expansion Workflow

Current Status

Metric	Count
Original dataset	108 policies
Vulnerable	41 (38%)
Secure	67 (62%)
Scraped policies	959 (unlabeled)
Target	800–1,000 labeled policies

Current Model Performance (Baseline)

Metric	Value
ROC-AUC	0.90
Accuracy (optimal threshold)	94.1%
Accuracy (default threshold 0.3)	~88%
F1-Score	54.5%
Recall	42.9%
Precision	75.0%

The low recall / F1 stem from the small dataset size (108 samples). Expanding the dataset should significantly improve these metrics.

Step-by-Step Process

Phase 1: Batch Scoring (Automated — ~3 minutes)

Run the trained model on all scraped policies to generate initial predictions.

```
# Score all 959 scraped policies
python scripts/label_scraped_policies.py \
    --scraped data/scraped_policies \
    --out data/labeled_policies \
    --model checkpoints/long/best_model.pt \
    --score-only
```

Expected output:

```
INFO Scoring 959 policies...
INFO Scoring complete: 959 scored, 0 skipped, 150.3 seconds
INFO Score distribution: vulnerable=~150, secure=~650, unknown=~159
```

What happens:
- Each policy is scored 0.0 (secure) to 1.0 (vulnerable)
- Policies with extractable JSON get model + heuristic scoring
- Policies without JSON get heuristic-only scoring
- Scores are saved to `data/labeled_policies/model_scores.json`

Phase 2: Interactive Review (Human — Sessions of ~1 hour)

Review policies in the terminal UI. Can quit and resume at any time.

```
# Start or resume interactive review
python scripts/label_scraped_policies.py \
    --scraped data/scraped_policies \
    --out data/labeled_policies \
    --review-only
```

Keyboard shortcuts:
- `a` = Accept prediction
- `r` = Reject (flip label)
- `e` = Edit severity
- `s` = Skip
- `q` = Quit & save

Tips for efficient reviewing:
1. Start with high-confidence predictions (>0.7 or <0.2) — most will be correct
2. Spend more time on mid-range scores (0.3–0.6) — these need human judgment
3. Look for PassRole + compute patterns (critical vulnerabilities)
4. Read-only policies (Describe, List, Get) are almost always secure
5. Wildcard resources with IAM actions are almost always vulnerable

Check progress anytime:

```
python -c "
import json
s = json.load(open('data/labeled_policies/labeling_state.json'))
print(f'Reviewed: {s[\"total_reviewed\"]}/{s[\"total_scored\"]}')
"
```

Phase 3: Merge Datasets

Combine original 108 policies with newly labeled policies.

```
# Dry run first (see stats without writing)
python scripts/merge_labeled_datasets.py \
    --original data/raw/samples \
    --new data/labeled_policies \
    --output data/raw/samples_expanded \
    --dry-run

# Actual merge
python scripts/merge_labeled_datasets.py \
    --original data/raw/samples \
    --new data/labeled_policies \
    --output data/raw/samples_expanded
```

Expected output:

```
============================================================
  Dataset Merge Summary
============================================================
  Original:              108 policies
    Vulnerable:           41
    Secure:               67
  New (labeled):         800 policies
    Vulnerable:          120
    Secure:              680
    Duplicates:            5 (removed)
  ----------------------------------
  Total merged:          903 policies
    Vulnerable:          161
    Secure:              747
============================================================
```

Phase 4: Retrain Model

Train on the expanded dataset.

```
# Update config to use expanded dataset
# Edit config.yaml: data_dir: data/raw/samples_expanded

# Retrain
policygraph train

# Evaluate
policygraph evaluate
```

Or train directly:

```
python scripts/train.py --data-dir data/raw/samples_expanded --epochs 50
python scripts/evaluate.py
```

Time Estimates

Task	Time	Can Be Parallelized?
Batch scoring (959 policies)	2–3 min	No (automated)
Review all 959 policies	~8 hours	Yes (multiple sessions)
Review 100 policies/session	~50 min	N/A
Dataset merge	< 1 min	No (automated)
Retraining on expanded data-set	5–15 min	No (GPU helps)
Total (all 959)	**~8–9 hours**	**Over 7–10 days**

Recommended Schedule

Day	Task	Policies Reviewed
Day 1	Batch scoring	0 (automated)
Day 2	Review high-confidence (>0.7, <0.2)	~200
Day 3–4	Review medium-confidence	~200
Day 5–7	Review remaining	~300
Day 8	Merge + retrain	-

Expected Improvements

With ~900 labeled policies (vs 108 currently):

Metric	Before (108)	After (~900)	Expected Gain
ROC-AUC	0.90	0.95+	+5–8%
Accuracy	88%	93–96%	+5–8%
F1-Score	54.5%	75–85%	+20–30%
Recall	42.9%	70–85%	+27–42%
Precision	75.0%	80–90%	+5–15%

Why these improvements?
- More diverse vulnerability patterns (not just 41 examples)
- Better class balance (more vulnerable samples)
- More negative examples help reduce false positives
- 8× more data enables better generalization
- Model can learn subtle patterns beyond the hand-crafted heuristics

Reducing heuristic dependence:
- Currently: 30% model + 70% heuristic (because dataset is too small)
- After expansion: Can shift to 70% model + 30% heuristic or even 100% model
- Adjust in `policygraph/analyzer.py` line 145:

```
python
  risk_score = max(0.0, min(1.0, 0.7 * model_risk_score + 0.3 * heuristic_risk_score))
```

File Locations

File	Purpose
`scripts/scrape_iam_terraform.py`	Scraper for GitHub Terraform configs
`scripts/label_scraped_policies.py`	Interactive labeling tool
`scripts/merge_labeled_datasets.py`	Dataset merge utility
`scripts/label_workflow.sh`	Quick-start commands reference
`data/scraped_policies/`	Raw scraped `.tf` files (959)
`data/labeled_policies/`	Labeled output (created during review)
`data/raw/samples/`	Original 108-policy dataset
`data/raw/samples_expanded/`	Merged dataset (created during merge)
`LABELING_GUIDE.md`	Detailed labeling instructions
`SCRAPING_REPORT.md`	Scraping run report